

Design and Implementation of a Database Management System for

Campus Placement and Recruitment Drive Management System

DBMS Course Project Report

Member	Roll number
Adarsh Tupsakri	25WU0102293
Akhil Uppula	25WU0102295
Akshay Konatham	25WU0101061

Course: Database Management Systems

Faculty: [Faculty name]

Section: [AIML Wolves / CSE Tigers / CSE Rhinos] | Project No. 22

Woxsen University, Hyderabad

Academic Year: 2026-27

2. Abstract

Every placement season, a college must coordinate students, recruiting companies, drives, job openings, interview rounds and offers. When this is tracked in spreadsheets and emails, eligibility errors, duplicate records and missing status information are common. This project designs and implements a relational database, *campus_placement*, in MySQL that stores this information in eight normalized (3NF) tables with primary keys, foreign keys and integrity constraints. A Flask REST API and a web interface connected to the database support inserting, viewing, updating and deleting records in all eight tables, and a built-in SQL console runs queries against the live database. SQL queries using joins, grouping and aggregate functions produce placement statistics such as applications per company, department-wise averages and overall placement percentage.

3. Introduction and Problem Statement

The Training and Placement Cell of a university manages many recruitment drives each year. Each company can run several drives; each drive offers one or more job roles with eligibility rules (minimum CGPA, maximum backlogs); students apply to roles, attend one or more interview rounds, and may receive an offer. The problem is to maintain this data accurately, prevent invalid or duplicate entries, and retrieve reports quickly. A DBMS with enforced constraints solves this far better than files or spreadsheets.

4. Objectives and Scope

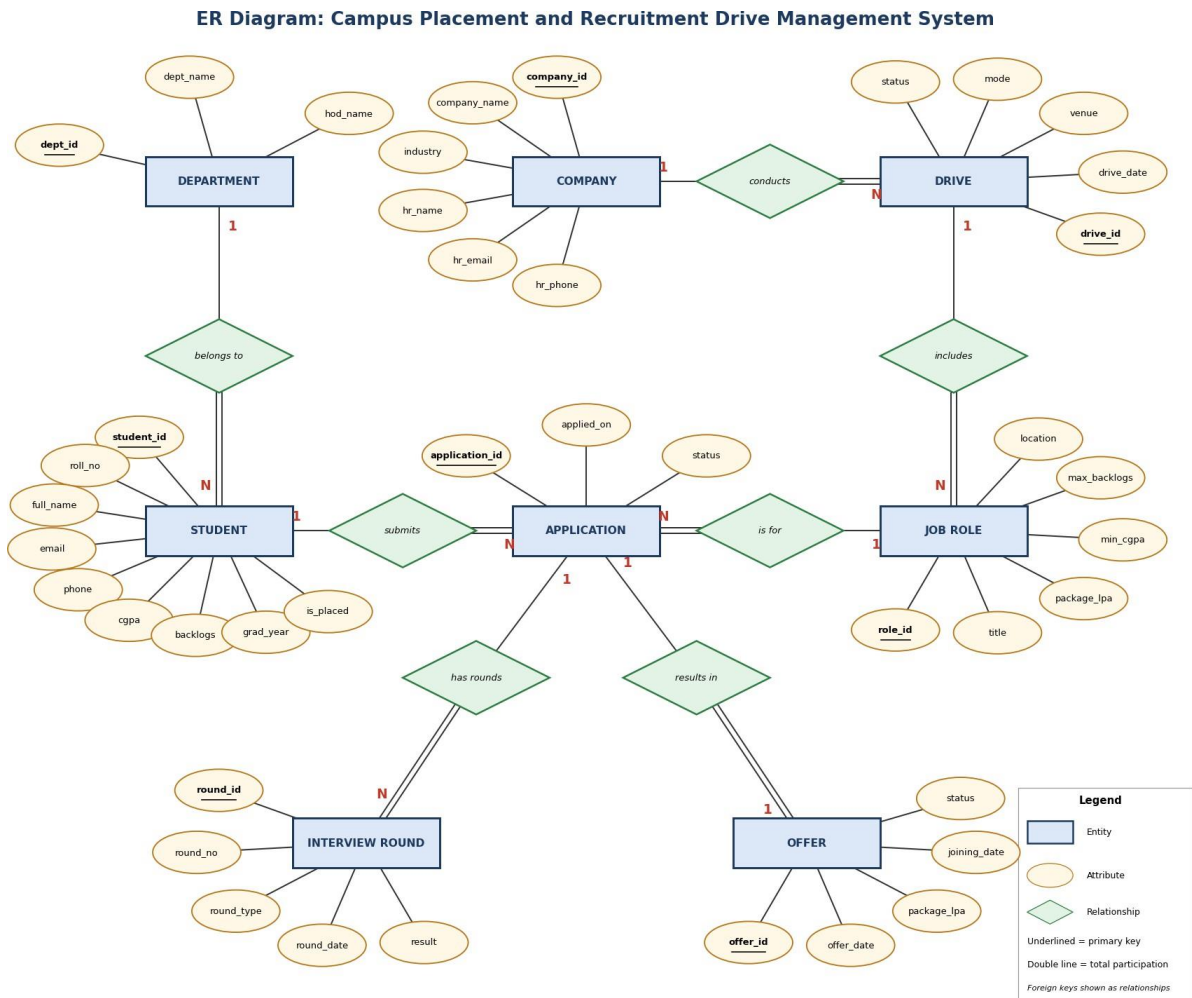
- Design an ER model for the placement process and map it to a relational schema in 3NF.
- Implement the database in MySQL with PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT and ENUM constraints.
- Provide SQL queries (joins, aggregates, subqueries) for placement reports.
- Build a web UI, backed by a Flask REST API, for inserting, viewing, updating and deleting records in all eight tables.

Scope: student, company, drive, role, application, interview-round and offer management for one institution. Out of scope: resume parsing, online tests, payments and multi-college support.

5. Software and Hardware Requirements

Item	Requirement
Database	MySQL 8.x (MySQL Workbench / CLI)
Backend	Python 3, Flask (REST API), mysql-connector-python
Frontend	Vite-built single-page web application served from the dist folder
Tools	Git and GitHub, VS Code
Hardware	Any PC with 4 GB RAM or more, 1 GB free disk, Windows / Linux / macOS

6. ER Diagram



The diagram uses Chen notation: rectangles are entities, ovals are attributes (underlined = primary key), diamonds are relationships and double lines show total participation. Foreign keys appear as relationships rather than attributes.

Relationships and cardinality: Department 1:N Student (total participation of student); Student 1:N Application; Job_Role 1:N Application; Company 1:N Placement_Drive; Placement_Drive 1:N Job_Role; Application 1:N Interview_Round; Application 1:0..1 Offer. A student may apply to many roles, and a role receives many applications, so Student and Job_Role have an M:N relationship resolved by the Application table.

7. Relational Schema and Normalization

department(dept_id, dept_name, hod_name)

student(student_id, roll_no, full_name, email, phone, dept_id, cgpa, backlogs, grad_year, is_placed)

company(company_id, company_name, industry, hr_name, hr_email, hr_phone)

placement_drive(drive_id, company_id, drive_date, venue, mode, status)

job_role(role_id, drive_id, title, package_lpa, min_cgpa, max_backlogs, location)

application(application_id, student_id, role_id, applied_on, status)

interview_round(round_id, *application_id*, round_no, round_type, round_date, result)

offer(offer_id, *application_id*, offer_date, package_lpa, joining_date, status)

(Underlined = primary key, italic = foreign key.)

1NF: all attributes are atomic and each table has a primary key. **2NF:** every non-key attribute depends on the whole key; for example, role details (title, package) are stored in job_role, not repeated in application. **3NF:** no transitive dependencies; department details (HOD) are stored once in department, and company HR details once in company. All functional dependencies have a candidate key on the left side, so the schema is also in BCNF.

8. Data Dictionary

Table: department

Column	Data type	Constraints
dept_id	INT	PK, AUTO_INCREMENT
dept_name	VARCHAR(60)	NOT NULL, UNIQUE
hod_name	VARCHAR(60)	NOT NULL

Table: student

Column	Data type	Constraints
student_id	INT	PK, AUTO_INCREMENT
roll_no	VARCHAR(15)	NOT NULL, UNIQUE
full_name	VARCHAR(80)	NOT NULL
email	VARCHAR(80)	NOT NULL, UNIQUE
phone	VARCHAR(15)	-
dept_id	INT	FK department, NOT NULL
cgpa	DECIMAL(3,2)	NOT NULL, CHECK 0 to 10
backlogs	INT	DEFAULT 0, CHECK >= 0
grad_year	YEAR	NOT NULL
is_placed	ENUM(No,Yes)	DEFAULT No

Table: company

Column	Data type	Constraints
company_id	INT	PK, AUTO_INCREMENT
company_name	VARCHAR(80)	NOT NULL, UNIQUE
industry	VARCHAR(50)	NOT NULL
hr_name	VARCHAR(60)	NOT NULL
hr_email	VARCHAR(80)	NOT NULL
hr_phone	VARCHAR(15)	-

Table: placement_drive

Column	Data type	Constraints
drive_id	INT	PK, AUTO_INCREMENT
company_id	INT	FK company, ON DELETE CASCADE
drive_date	DATE	NOT NULL
venue	VARCHAR(60)	NOT NULL
mode	ENUM	On-campus / Off-campus / Virtual
status	ENUM	Scheduled / Completed / Cancelled, DEFAULT Scheduled

Table: job_role

Column	Data type	Constraints
role_id	INT	PK, AUTO_INCREMENT
drive_id	INT	FK placement_drive, CASCADE
title	VARCHAR(60)	NOT NULL
package_lpa	DECIMAL(5,2)	NOT NULL, CHECK > 0
min_cgpa	DECIMAL(3,2)	DEFAULT 6.00
max_backlogs	INT	DEFAULT 0
location	VARCHAR(40)	NOT NULL

Table: application

Column	Data type	Constraints
application_id	INT	PK, AUTO_INCREMENT
student_id	INT	FK student, CASCADE
role_id	INT	FK job_role, CASCADE
applied_on	DATE	NOT NULL
status	ENUM	Applied / Shortlisted / Rejected / Selected
(student_id, role_id)	-	UNIQUE

Table: interview_round

Column	Data type	Constraints
round_id	INT	PK, AUTO_INCREMENT
application_id	INT	FK application, CASCADE
round_no	INT	NOT NULL
round_type	ENUM	Aptitude / Technical / HR / Group Discussion
round_date	DATE	NOT NULL
result	ENUM	Pending / Cleared / Failed, DEFAULT Pending
(application_id, round_no)	-	UNIQUE

Table: offer

Column	Data type	Constraints
offer_id	INT	PK, AUTO_INCREMENT

Column	Data type	Constraints
application_id	INT	FK application, UNIQUE, CASCADE
offer_date	DATE	NOT NULL
package_lpa	DECIMAL(5,2)	NOT NULL, CHECK > 0
joining_date	DATE	-
status	ENUM	Offered / Accepted / Declined, DEFAULT Offered

9. SQL Commands Used (DDL and DML)

DDL: database and table creation

```

-- ----- DDL -----
CREATE TABLE department (
    dept_id      INT AUTO_INCREMENT PRIMARY KEY,
    dept_name    VARCHAR(60) NOT NULL UNIQUE,
    hod_name     VARCHAR(60) NOT NULL
);

CREATE TABLE student (
    student_id   INT AUTO_INCREMENT PRIMARY KEY,
    roll_no      VARCHAR(15) NOT NULL UNIQUE,
    full_name    VARCHAR(80) NOT NULL,
    email        VARCHAR(80) NOT NULL UNIQUE,
    phone        VARCHAR(15),
    dept_id      INT NOT NULL,
    cgpa         DECIMAL(3,2) NOT NULL CHECK (cgpa BETWEEN 0 AND 10),
    backlogs     INT NOT NULL DEFAULT 0 CHECK (backlogs >= 0),
    grad_year    YEAR NOT NULL,
    is_placed    ENUM('No','Yes') NOT NULL DEFAULT 'No',
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE company (
    company_id   INT AUTO_INCREMENT PRIMARY KEY,
    company_name VARCHAR(80) NOT NULL UNIQUE,
    industry     VARCHAR(50) NOT NULL,
    hr_name      VARCHAR(60) NOT NULL,
    hr_email     VARCHAR(80) NOT NULL,
    hr_phone     VARCHAR(15)
);

CREATE TABLE placement_drive (
    drive_id     INT AUTO_INCREMENT PRIMARY KEY,
    company_id   INT NOT NULL,
    drive_date   DATE NOT NULL,
    venue        VARCHAR(60) NOT NULL,
    mode         ENUM('On-campus','Off-campus','Virtual') NOT NULL,
    status       ENUM('Scheduled','Completed','Cancelled') NOT NULL DEFAULT 'Scheduled',
    FOREIGN KEY (company_id) REFERENCES company(company_id) ON DELETE CASCADE
);

CREATE TABLE job_role (
    role_id      INT AUTO_INCREMENT PRIMARY KEY,
    drive_id     INT NOT NULL,
    title        VARCHAR(60) NOT NULL,
    package_lpa  DECIMAL(5,2) NOT NULL CHECK (package_lpa > 0),
    min_cgpa     DECIMAL(3,2) NOT NULL DEFAULT 6.00,
    max_backlogs INT NOT NULL DEFAULT 0,
    location     VARCHAR(40) NOT NULL,
    FOREIGN KEY (drive_id) REFERENCES placement_drive(drive_id) ON DELETE CASCADE
);

CREATE TABLE application (
    application_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id     INT NOT NULL,
    role_id        INT NOT NULL,
    applied_on     DATE NOT NULL,
    status         ENUM('Applied','Shortlisted','Rejected','Selected') NOT NULL DEFAULT 'Applied',
    UNIQUE (student_id, role_id),

```

```

FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE,
FOREIGN KEY (role_id) REFERENCES job_role(role_id) ON DELETE CASCADE
);

CREATE TABLE interview_round (
    round_id INT AUTO_INCREMENT PRIMARY KEY,
    application_id INT NOT NULL,
    round_no INT NOT NULL,
    round_type ENUM('Aptitude','Technical','HR','Group Discussion') NOT NULL,
    round_date DATE NOT NULL,
    result ENUM('Pending','Cleared','Failed') NOT NULL DEFAULT 'Pending',
    UNIQUE (application_id, round_no),
    FOREIGN KEY (application_id) REFERENCES application(application_id) ON DELETE CASCADE
);

CREATE TABLE offer (
    offer_id INT AUTO_INCREMENT PRIMARY KEY,
    application_id INT NOT NULL UNIQUE,
    offer_date DATE NOT NULL,
    package_lpa DECIMAL(5,2) NOT NULL CHECK (package_lpa > 0),
    joining_date DATE,
    status ENUM('Offered','Accepted','Declined') NOT NULL DEFAULT 'Offered',
    FOREIGN KEY (application_id) REFERENCES application(application_id) ON DELETE CASCADE
);

```

DML: sample data insertion (minimum 5 records per table)

```

-- ----- DML: sample data (min. 5 rows per table) -----
INSERT INTO department (dept_name, hod_name) VALUES
('CSE', 'Dr. R. Menon'), ('CSE (AI & ML)', 'Dr. S. Iyer'),
('ECE', 'Dr. K. Rao'), ('Mechanical', 'Dr. P. Reddy'), ('Civil', 'Dr. A. Sharma');

INSERT INTO student (roll_no, full_name, email, phone, dept_id, cgpa, backlogs, grad_year, is_placed) VALUES
('21WU0101001', 'Ananya Reddy', 'ananya.r@woxsen.edu.in', '9876500001', 2, 9.10, 0, 2026, 'Yes'),
('21WU0101002', 'Rahul Verma', 'rahul.v@woxsen.edu.in', '9876500002', 1, 8.40, 0, 2026, 'Yes'),
('21WU0101003', 'Sneha Kulkarni', 'sneha.k@woxsen.edu.in', '9876500003', 1, 7.60, 1, 2026, 'No'),
('21WU0101004', 'Imran Shaikh', 'imran.s@woxsen.edu.in', '9876500004', 3, 6.90, 0, 2026, 'No'),
('21WU0101005', 'Priya Nair', 'priya.n@woxsen.edu.in', '9876500005', 2, 8.80, 0, 2026, 'No'),
('21WU0101006', 'Karthik Rao', 'karthik.r@woxsen.edu.in', '9876500006', 4, 6.20, 2, 2026, 'No'),
('21WU0101007', 'Meera Joshi', 'meera.j@woxsen.edu.in', '9876500007', 5, 7.20, 0, 2026, 'No');

INSERT INTO company (company_name, industry, hr_name, hr_email, hr_phone) VALUES
('TechNova Solutions', 'IT Services', 'Aarti Mehta', 'hr@technova.com', '9000011111'),
('DataBridge Analytics', 'Analytics', 'Vikram Singh', 'talent@databridge.io', '9000022222'),
('Infracore Constructions', 'Construction', 'Suresh Patil', 'careers@infracore.in', '9000033333'),
('AutoMech Industries', 'Manufacturing', 'Neha Gupta', 'hr@automech.com', '9000044444'),
('FinEdge Capital', 'Finance', 'Rohit Das', 'jobs@finedge.com', '9000055555');

INSERT INTO placement_drive (company_id, drive_date, venue, mode, status) VALUES
(1, '2026-08-10', 'Auditorium A', 'On-campus', 'Completed'),
(2, '2026-08-24', 'Seminar Hall 2', 'On-campus', 'Completed'),
(3, '2026-09-05', 'Civil Block', 'On-campus', 'Completed'),
(4, '2026-10-15', 'Auditorium B', 'On-campus', 'Scheduled'),
(5, '2026-10-22', 'Online', 'Virtual', 'Scheduled');

INSERT INTO job_role (drive_id, title, package_lpa, min_cgpa, max_backlogs, location) VALUES
(1, 'Software Engineer', 6.50, 7.00, 0, 'Hyderabad'),
(1, 'Systems Analyst', 5.00, 6.50, 1, 'Pune'),
(2, 'Data Analyst', 8.00, 8.00, 0, 'Bengaluru'),
(3, 'Site Engineer', 4.50, 6.00, 1, 'Hyderabad'),
(4, 'Design Engineer', 5.50, 6.50, 0, 'Chennai'),
(5, 'Financial Analyst', 9.00, 8.00, 0, 'Mumbai');

INSERT INTO application (student_id, role_id, applied_on, status) VALUES
(1, 3, '2026-08-12', 'Selected'),
(2, 1, '2026-08-05', 'Selected'),
(3, 2, '2026-08-06', 'Selected'),
(4, 1, '2026-08-05', 'Rejected'),
(5, 3, '2026-08-12', 'Rejected'),
(5, 6, '2026-10-01', 'Applied'),
(7, 4, '2026-08-28', 'Selected'),
(1, 6, '2026-10-01', 'Applied'),
(4, 2, '2026-08-06', 'Selected');

INSERT INTO interview_round (application_id, round_no, round_type, round_date, result) VALUES

```

```
(1,1,'Aptitude','2026-08-24','Cleared'),
(1,2,'Technical','2026-08-24','Cleared'),
(1,3,'HR','2026-08-25','Cleared'),
(2,1,'Technical','2026-08-10','Cleared'),
(2,2,'HR','2026-08-10','Cleared'),
(3,1,'Aptitude','2026-08-10','Cleared'),
(3,2,'Technical','2026-08-11','Pending'),
(4,1,'Aptitude','2026-08-10','Failed'),
(5,1,'Technical','2026-08-24','Failed');
```

```
INSERT INTO offer (application_id, offer_date, package_lpa, joining_date, status) VALUES
(1,'2026-08-27',8.00,'2027-07-01','Accepted'),
(2,'2026-08-12',6.50,'2027-07-15','Accepted'),
(7,'2026-09-10',4.50,'2027-08-01','Offered'),
(3,'2026-08-14',5.00,'2027-08-01','Declined'),
(9,'2026-08-15',5.00,'2027-08-01','Offered');
```

Sample output: row counts

```
+-----+-----+
| t      | n |
+-----+-----+
| department | 5 |
| student    | 7 |
| company    | 5 |
| placement_drive | 5 |
| job_role   | 6 |
| application | 9 |
| interview_round | 9 |
| offer      | 5 |
+-----+-----+
```

UPDATE and DELETE

```
UPDATE application SET status = 'Selected' WHERE application_id = 3;
DELETE FROM application WHERE application_id = 8; -- cascades to interview_round and offer
```

10. Queries with Outputs

Q1: Students with their department

```
SELECT s.roll_no, s.full_name, d.dept_name, s.cgpa
FROM student s JOIN department d ON s.dept_id = d.dept_id
ORDER BY s.cgpa DESC;
```

```
+-----+-----+-----+-----+
| roll_no | full_name | dept_name | cgpa |
+-----+-----+-----+-----+
| 21WU0101001 | Ananya Reddy | CSE (AI & ML) | 9.10 |
| 21WU0101005 | Priya Nair | CSE (AI & ML) | 8.80 |
| 21WU0101002 | Rahul Verma | CSE | 8.40 |
| 21WU0101003 | Sneha Kulkarni | CSE | 7.60 |
| 21WU0101007 | Meera Joshi | Civil | 7.20 |
| 21WU0101004 | Imran Shaikh | ECE | 6.90 |
| 21WU0101006 | Karthik Rao | Mechanical | 6.20 |
+-----+-----+-----+-----+
```

Q2: Drives with company and roles on offer

```
SELECT c.company_name, pd.drive_date, pd.mode, jr.title, jr.package_lpa
FROM placement_drive pd
JOIN company c ON pd.company_id = c.company_id
JOIN job_role jr ON jr.drive_id = pd.drive_id
ORDER BY pd.drive_date;
```

```
+-----+-----+-----+-----+-----+
| company_name | drive_date | mode | title | package_lpa |
+-----+-----+-----+-----+-----+
| TechNova Solutions | 2026-08-10 | On-campus | Software Engineer | 6.50 |
| TechNova Solutions | 2026-08-10 | On-campus | Systems Analyst | 5.00 |
| DataBridge Analytics | 2026-08-24 | On-campus | Data Analyst | 8.00 |
| Infracore Constructions | 2026-09-05 | On-campus | Site Engineer | 4.50 |
```

AutoMech Industries	2026-10-15	On-campus	Design Engineer	5.50
FinEdge Capital	2026-10-22	Virtual	Financial Analyst	9.00

Q3: Number of applications per company (aggregate + GROUP BY)

```
SELECT c.company_name, COUNT(a.application_id) AS total_applications
FROM company c
JOIN placement_drive pd ON pd.company_id = c.company_id
JOIN job_role jr ON jr.drive_id = pd.drive_id
LEFT JOIN application a ON a.role_id = jr.role_id
GROUP BY c.company_name
ORDER BY total_applications DESC;
```

company_name	total_applications
TechNova Solutions	4
DataBridge Analytics	2
FinEdge Capital	2
Infracore Constructions	1
AutoMech Industries	0

Q4: Department-wise placement count and average package

```
SELECT d.dept_name, COUNT(DISTINCT s.student_id) AS students_with_offers,
ROUND(AVG(o.package_lpa),2) AS avg_package_lpa
FROM offer o
JOIN application a ON o.application_id = a.application_id
JOIN student s ON a.student_id = s.student_id
JOIN department d ON s.dept_id = d.dept_id
WHERE o.status = 'Accepted'
GROUP BY d.dept_name;
```

dept_name	students_with_offers	avg_package_lpa
CSE	1	6.50
CSE (AI & ML)	1	8.00

Q5: Highest, lowest and average package overall

```
SELECT MAX(package_lpa) AS highest, MIN(package_lpa) AS lowest,
ROUND(AVG(package_lpa),2) AS average
FROM offer WHERE status IN ('Offered','Accepted');
```

highest	lowest	average
8.00	4.50	6.00

Q6: Eligible unplaced students for a role (CGPA and backlog criteria)

```
SELECT s.full_name, s.cgpa, s.backlogs, jr.title
FROM student s JOIN job_role jr
ON s.cgpa >= jr.min_cgpa AND s.backlogs <= jr.max_backlogs
WHERE s.is_placed = 'No' AND jr.title = 'Financial Analyst';
```

full_name	cgpa	backlogs	title
Priya Nair	8.80	0	Financial Analyst

Q7: Placement percentage (subquery)

```
SELECT ROUND(100 * SUM(is_placed = 'Yes') / COUNT(*), 1) AS placement_percent FROM student;
```

placement_percent

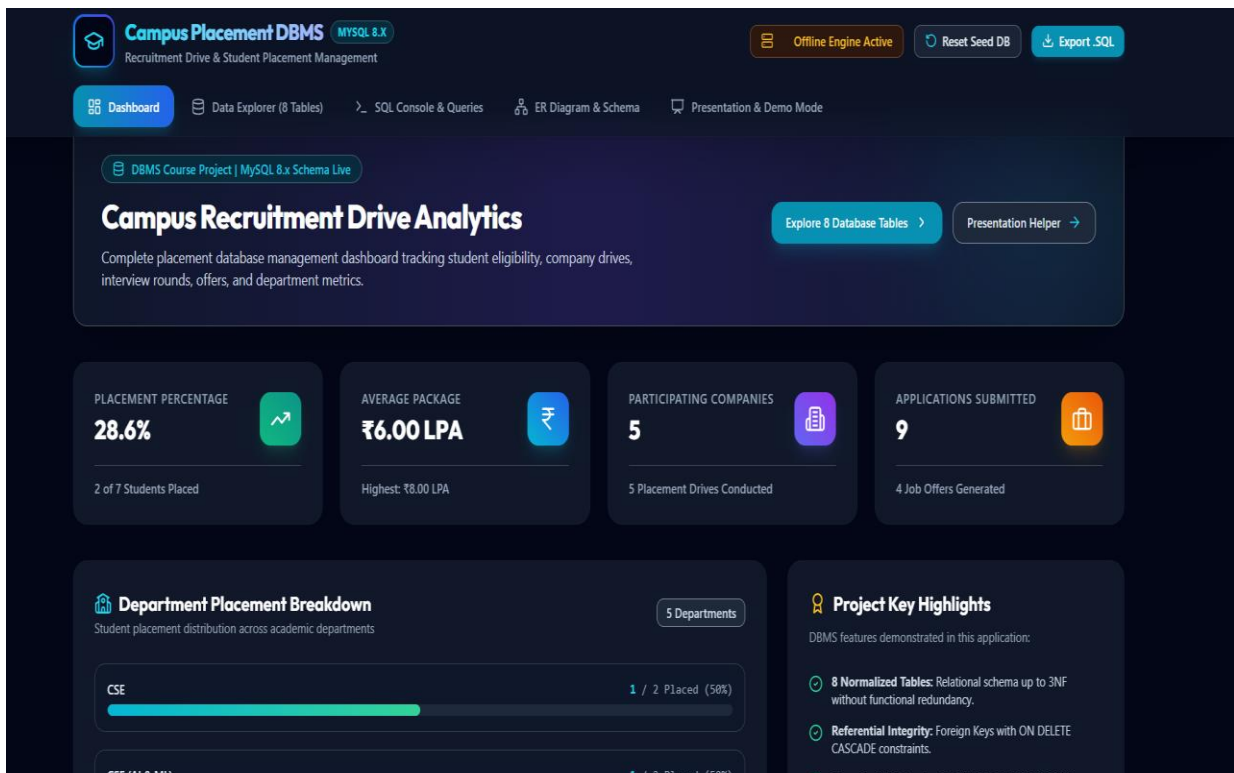
```

+-----+
|                |
|                | 28.6 |
|                |
+-----+

```

11. UI Design and Screenshots

The interface is a Vite-built single-page web application that talks to the Flask backend through JSON endpoints. Through these endpoints it supports: a dashboard with the number of records in each of the eight tables; viewing every table, with joined columns such as department, company and student names in place of raw IDs; inserting, updating and deleting records in every table; a SQL console that runs queries against the live database; and a database connection panel that shows connection status.



 Campus Placement DBMS MySQL 8.0 Recruitment Drive & Student Placement Management Offline Engine Active Reset Seed DB Export SQL											
Dashboard Data Explorer (8 Tables) SQL Console & Queries ER Diagram & Schema Presentation & Demo Mode											
BACKLOGS	CGPA	DEPT ID	DEPT NAME	EMAIL	FULL NAME	GRAD YEAR	IS PLACED	PHONE	ROLL NO	STUDENT ID	ACTIONS
0	7.20	5	Civil	meera.j@woxsen.edu.in	Meera Joshi	2026	No	9876500007	21WU0101007	7	 
2	6.20	4	Mechanical	karthik.r@woxsen.edu.in	Karthik Rao	2026	No	9876500006	21WU0101006	6	 
0	8.80	2	CSE (AI & ML)	priya.n@woxsen.edu.in	Priya Nair	2026	No	9876500005	21WU0101005	5	 
0	6.90	3	ECE	imran.s@woxsen.edu.in	Imran Shaikh	2026	No	9876500004	21WU0101004	4	 
1	7.60	1	CSE	sneha.k@woxsen.edu.in	Sneha Kulkarni	2026	No	9876500003	21WU0101003	3	 
0	8.40	1	CSE	rahul.v@woxsen.edu.in	Rahul Verma	2026	Yes	9876500002	21WU0101002	2	 
0	9.10	2	CSE (AI & ML)	ananya.r@woxsen.edu.in	Ananya Reddy	2026	Yes	9876500001	21WU0101001	1	 

Campus Placement & Recruitment Drive Management System | DBMS Course Project 2026

12. Implementation Details

Technology stack: MySQL 8 (database), Python 3 and Flask (REST API backend), mysql-connector-python (connectivity) and a Vite-built single-page application (front end, served by Flask from the dist folder). Every statement that carries user values is parameterised (%s placeholders), which prevents SQL injection, and table names are checked against a fixed list of the eight real tables before they are used in a query. Connection settings are read from environment variables, with defaults for local use.

Endpoint	Method	Purpose
/api/config	GET, POST	Show or change connection settings; reports connection status
/api/tables	GET	Record count of each table (dashboard)
/api/table/<name>	GET	View all rows, with joins for readable names
/api/table/<name>	POST	Insert a record
/api/table/<name>/<id>	PUT	Update a record
/api/table/<name>/<id>	DELETE	Delete a record (dependent rows removed by CASCADE)
/api/query/execute	POST	Run a SQL statement from the console
/api/seed	POST	Rebuild tables and sample data from schema.sql

```
DB_CONFIG = {"host": os.environ.get("MYSQL_HOST", "localhost"), "port": int(os.environ.get("MYSQL_PORT", 3306)),
             "user": os.environ.get("MYSQL_USER", "root"), "password": os.environ.get("MYSQL_PASSWORD", ""),
             "database": os.environ.get("MYSQL_DATABASE", "campus_placement"), "autocommit": True}
PRIMARY_KEYS = {"department": "dept_id", "student": "student_id", "company": "company_id",
                "placement_drive": "drive_id", "job_role": "role_id", "application": "application_id",
                "interview_round": "round_id", "offer": "offer_id"}

@app.route("/api/table/<table_name>", methods=["POST"])
def insert_record(table_name):
    if table_name not in PRIMARY_KEYS:
        return jsonify({"success": False, "error": "Invalid table"}), 400
    data = request.json or {}
    columns = list(data.keys())
    query = f"INSERT INTO `{table_name}` ({', '.join('`'+c+'`' for c in columns)}) VALUES ({', '.join(['%s']*len(columns))})"
    cursor.execute(query, [data[c] for c in columns])

@app.route("/api/table/<table_name>/<int:record_id>", methods=["DELETE"])
def delete_record(table_name, record_id):
    pk = PRIMARY_KEYS[table_name]
    cursor.execute(f"DELETE FROM `{table_name}` WHERE `{pk}` = %s", (record_id,))
```

13. Testing

No.	Test case	Result observed	Status
1	Insert a record in each table through the UI	Row appears in the table view and in MySQL Workbench	[tick after testing]
2	Delete an application through the UI	Application, its interview rounds and its offer are removed (CASCADE)	[tick after testing]
3	Update a record status through the UI	Change is visible in Workbench	[tick after testing]
4	Duplicate application (same student and role)	ERROR 1062 (23000) at line 1: Duplicate entry '1-3' for key 'student_id'	Pass (rejected)
5	CGPA = -1 (CHECK constraint)	ERROR 4025 (23000) at line 1: CONSTRAINT `student.cgpa` failed for `campus_placement`.`student`	Pass (rejected)
6	Student with unknown department	ERROR 1452 (23000) at line 1: Cannot add or update a child row: a foreign key constraint fails (`campus_placement`.`student`, CONS	Pass (rejected)
7	Duplicate email	ERROR 1062 (23000) at line 1: Duplicate entry 'ananya.r@woxsen.edu.in' for key 'email'	Pass (rejected)

14. Conclusion and Future Enhancements

The system stores the complete placement lifecycle, from company drive to student offer, in a normalized MySQL database whose constraints prevent duplicate applications, invalid CGPA values and orphan records. The UI demonstrates insertion, deletion and viewing against the live database.

Future enhancements: authentication and role-based access (students, recruiters, placement officer), and restricting the SQL console to administrators; resume upload; automatic email notification of shortlists; a trigger to update *is_placed* when an offer is accepted; an analytics dashboard with charts; and an automatic eligibility check at application time.

15. References

- Elmasri, R. and Navathe, S., Fundamentals of Database Systems, Pearson.
- Silberschatz, A., Korth, H. and Sudarshan, S., Database System Concepts, McGraw-Hill.
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/>
- Flask Documentation, <https://flask.palletsprojects.com/>
- mysql-connector-python Documentation, <https://dev.mysql.com/doc/connector-python/>

16. Contribution of Each Member

Member	Contribution
Adarsh Tupsakri (25WU0102293)	Developed the database (schema, sample data, queries), testing, and front end (contributor 2)
Akhil Uppula (25WU0102295)	Flask backend (REST API and database connectivity)
Akshay Konatham (25WU0101061)	ER diagram and front end (contributor 1)

17. Appendix: GitHub Repository

Repository: <https://github.com/AdarshTupsakri/DBMS-COURSEPROJECT>

Folders: Presentation, Project-Report